Using RL for Traffic Signal Control

Mingjie Gao, Xiao Sun, Tianrui Luo, Zhiqiang Yin
Department of Electrical & Computer Engineering
University of Toronto
{mingjie.gao, xiaocn.sun, tianrui.luo, zhiqiang.yin}@mail.utoronto.ca

Abstract

Traffic congestion, caused by static signal control, is a critical urban challenge. This paper investigates an adaptive traffic signal control system using Deep Reinforcement Learning (DRL), formulated as an MDP to maximize throughput and minimize waiting time in a custom intersection simulation. We compare DQN and PPO agents against Fixed-Timing and Round Robin baselines, utilizing ablation studies on reward composition, environment realism, and signal-switch delay (PPO). Results show DRL controllers, particularly DQN, outperform static baselines in throughput and waiting time. Adaptive models performed better in high-density, realistic traffic, emphasizing dynamic control necessity. The PPO study found a moderate signal-switch delay optimizes the throughput-waiting time trade-off. A stable-incremental reward function proved essential for convergence stability. This work validates DRL as a robust solution for dynamic traffic management, highlighting the critical role of reward structure and operational constraints.

Assentation of Teamwork

Mingjie Gao designed the traffic environment and visualization, implemented the FixedTiming controller, and developed the DQN algorithm. Additionally, Mingjie was responsible for editing the reports.

Xiao Sun contributed to the development and evolution of PPO algorithm and comparison with round robin baseline. Xiao also edited and formatted the reports properly.

Tianrui Luo refined the Traffic visualization and designed the FixedTiming controller. Tianrui also added ablation experiments and tail analysis for DQN & Fixed-Timing models, and edited the README section and reports.

Zhiqiang Yin implemented the round robin baseline controller and the PPO algorithm. Zhiqiang also explored the ablation experiments about the delay of signal switch and rewarding shaping for PPO & round-robin baseline model, and wrote the reports about the Methodology, ablation experiment, and discussion section related to round-robin and PPO.

1 Introduction

The problem of traffic congestion has been a major challenge for many modern cities. In urban areas, traditional signal control strategies are usually static and unable to adapt to the real-time situation of the traffic, causing non-optimal performance when the traffic volumes change during the day.

Reinforcement Learning provides a data-driven framework to develop adaptive traffic signal control systems that learn and evolve optimal signal policies. This project uses RL to solve the traffic signal control system problem with a lightweight simulation environment. The goal is to demonstrate how

RL can help improve the traffic management performance while indicating the trade-offs between model complexity, generalization, and applicability in real world.

2 Preliminaries and Problem Formulation

Traffic signal control is modeled as a Markov decision process (MDP) defined by (S, A, P, R, gamma). The state (S) consists of real-time traffic metrics (e.g., queue length, vehicle distance). The action space (A) involves switching signal phases, and the reward (R), defined as the negative total traffic delay, guides the agent to minimize efficiency loss.The primary goal is to develop an adaptive RL policy to maximize throughput and minimize average vehicle wait time at a single urban intersection. Due to the limitations of existing platforms like Highway-env (1), we constructed a custom four-way intersection environment1. This environment was simplified to control only randomly generated straight and left-turn traffic flows (two lanes per direction), excluding right-turn traffic and the yellow light transition phase.To evaluate performance, we compare two DRL agents (Deep Q-Network, DQN, and Proximal Policy Optimization, PPO) with two traditional benchmarks (Fixed-Time and Round Robin models). The final comparison uses key metrics, specifically average vehicle waiting time and average throughput, within our custom simulation environment.

3 Design

3.1 Baseline

The goal of building up the model is used as performance reference when we design and test our RL agent. We have selected two methods (Fixed Time, Round Robin) to build up baseline model.

Fixed Time: We first estimate the traffic flow and predefined each traffic light duration time. Each current phrase (traffic light) will change to the next current phrase at a constant time. The transfer of the current phrase repeats in a cycle, regardless how the environment has changed.

Round Robin: We implement a simple round-robin controller as a non-adaptive baseline. The intersection alternates between the East–West and North–South phases in a fixed cyclic order, assigning equal green durations to each phase. This ensures fairness between the two directions but does not respond to varying traffic densities.

3.2 Reinforcement Agent

Deep-Q Network (DQN) (2)is a popular off-policy reinforcement learning algorithm which estimates the action-value function (q(s,a)) by neural network and chooses subsequent actions that maximize the return. DQN has two core mechanisms - experience replay, which breaks the potential correlation between samplings by random selection, and target network, which utilizes two different neural networks to prevent overfitting and gradual parameter updates.The architecture of Double DQN and Dueling head will ensure algorithm stability as well. This algorithm is ideal for our traffic signal control problem due to its nature of effectiveness for discrete action choice problems.

Proximal Policy Optimization (PPO) is a popular on-policy reinforcement learning algorithm which optimizes the policy by gradient estimates of value function. PPO has a core mechanism of clipping, which limits the step size in policy iteration. By constraining the policy change ratio in each update, PPO ensures steady policy improvement. In addition, Generalized Advantage Estimation (GAE) will also be implemented to calculate the reward. This algorithm is ideal for our traffic signal control problem due to its capability of hybrid/ continuous control and stable delivery of learning curve.

4 Methodology

4.1 Traffic Environment Setup & Visualization

The experimental foundation is a custom-built, discrete-time traffic simulation environment modeling a four-way signalized intersection, providing the dynamic platform for training RL agents. Each approach has a straight-through lane and a left-turn lane. The simulation manages vehicle dynamics (acceleration/deceleration, stopping at red). Left-turn movements are simplified to an instantaneous

turn when the vehicle clears the centerline under a green light. Traffic demand supports dynamic flow; the environment also simulates distinct Off-Peak and Peak-Hour periods by varying vehicle spawn probabilities.

4.2 Baseline Model - Fixed-Timing

Fixed timing is the most traditional traffic control strategy, characterized by its reliance on pre-set static signal phase cycles, where the duration of each phase remains constant. In our simulation environment, the fixed-timing controller operates in a four-phase cyclic manner to manage straight-ahead and left-turn traffic at an intersection: N-S straight-ahead, E-W straight-ahead, N-S left-turn, E-W left-turn. The controller executes strictly in this order, and will only transition to the next stage after the current stage's allocation time has expired.

4.3 Baseline Model - Round Robin

We use a round-robin controller as our primary baseline. In this strategy, the intersection cycles through all signal phases in a fixed order, and each phase is held for a constant duration. The switching schedule is completely independent of the real-time traffic state (queue lengths, waiting times, or arrival rates). Concretely, in our 4-phase setting, the controller activates phase 0 for a number of X steps, then phase 1 for a number of X steps, and so on through phases 2 and 3 before returning to phase 0, repeating this cycle indefinitely.

4.4 RL Model - DQN

The traffic signal control problem is formalized as a Markov Decision Process (MDP), defining three components in DQN application: state, action, and reward. The state is a normalized 20-dimensional vector encompassing the current traffic conditions: queue lengths, nearest vehicle distances for the eight lanes, and a four-dimensional one-hot encoding of the current phase. The action is a discrete selection from four available traffic phases, satisfying the minimum phase duration constraint before execution. The reward function aims to maximize throughput and minimize waiting time.

The DQN model utilizes a PyTorch-implemented, standard feed-forward architecture. It processes the 20-dimensional state vector through a first layer (output 256, ReLU) and a subsequent hidden layer (output 128, ReLU). The final output layer takes the 128-dimensional input and produces a 4-dimensional vector, estimating the Q-value for each action using a Linear activation.

The DQN agent employs several crucial techniques for stable and accelerated learning. The Experience Replay Buffer stores transitions and samples uncorrelated batches, stabilizing learning by breaking temporal correlation. A key enhancement is Double DQN (3), which addresses overestimation bias by decoupling action selection and value estimation using separate Online and Target Networks. The Target Network parameters are managed via Soft Target Update, periodically synchronized for stable targets. Action selection uses the epsilon-Greedy Exploration strategy, balancing exploration and exploitation, with epsilon decaying linearly over time. Procedural Enhancements include a 10,000-step Warm-up Phase to populate the replay buffer before training begins. Additionally, learning rate is scheduled to decay mid-training (e.g., at episode 300) to 5e-5, and epsilon value is periodically reset to 0.3 (e.g., every 150 episodes) to encourage renewed exploration and prevent local optima. Optimization utilizes the Adam optimizer combined with Smooth L1 Loss (Huber Loss) for robustness against outliers. Gradient Clipping is also applied to prevent exploding gradients.

4.5 RL Model - PPO

We use Proximal Policy Optimization (PPO) as our learning-based controller in an actor–critic framework. The goal is to learn a stochastic policy $\pi_\theta(a \mid s)$ that maps the current traffic state s_t to a probability distribution over traffic signal phases a_t.

The **actor** is a policy network implementing $\pi_\theta(a \mid s)$. It maps the traffic state s_t to a probability distribution over the four possible signal phases a_t. The **critic** is a value function network $V_\phi(s)$ that estimates the expected return at state s under the current policy. The return R_t is defined as the discounted sum of future rewards sampled from the environment: $R_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k}$, where r_{t+k} is the reward at time $t + k$ and $\gamma \in (0, 1)$ is the discount factor. The critic is trained to approximate

the expected return under the current policy, $V_\phi(s_t) \approx \mathbb{E}[R_t \mid s_t]$, by minimizing a squared-error loss: $L_{value}(\phi) = \mathbb{E}_t \left[\left(V_\phi(s_t) - R_t \right)^2 \right]$.

To update the policy, we need an estimate of how good an action was relative to the critic's baseline, we use Generalized Advantage Estimation (GAE). First, we define the temporal-difference (TD) error as $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$. Then the advantage estimate is $\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \, \delta_{t+l}$, where $\lambda \in [0, 1]$ controls the bias–variance trade-off. Intuitively, $\hat{A}_t > 0$ indicates that the action at time step t performed better than expected, while $\hat{A}_t < 0$ indicates it performed worse.

Let the probability ratio between the new policy and the old policy be $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$.

The PPO clipped surrogate objective is given by

$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \, A_t, \; clip(r_t(\theta), \, 1 - \epsilon, \, 1 + \epsilon) \, A_t \right) \right]$, which limits the ratio to $[1-\epsilon, \, 1+\epsilon]$.

The min and clipping prevent r(t) from moving too far away from 1, which stabilizes training by avoiding overly large policy updates compared with old policy. To encourage exploration, we also add an entropy bonus term $L^{entropy} = \mathbb{E}_t \left[- \sum_a \pi_\theta(a \mid s_t) \, \log \pi_\theta(a \mid s_t) \right]$.

The final loss we maximize combines all three terms: $L(\theta, \phi) = L_{CLIP}(\theta) - c_v L_{value}(\phi) + c_{entropy} L_{entropy}(\theta)$, with c_v: the value loss coefficient and $c_{entropy}$: the entropy coefficient. Maximizing this loss corresponds to maximizing the clipped policy objective and entropy, minimizing the penalty of the critic to returns.

In our implementation, we train PPO directly on the modified traffic environment. At each iteration, we run the current policy for a fixed number of environment steps, store the transitions, and then perform several epochs of gradient updates on minibatches of this data. A high-level pseudocode description is included in Appendix 2.

5 Numerical Experiments

In order to test the stability and performance of our RL models compared with the baseline models, we proposed several ablation experiments, aiming to compare the influence of different configurations on model performance. For numeric illustration of traffic efficiency, we used average traffic throughput and waiting time for the last 30% episodes during testing.

5.1 Performance on Realistic v.s. Ideal Traffic Environment - DQN & fixed time baseline

For both baseline and RL models, we introduced two traffic environments to simulate ideal and realistic situations, where the latter contains noises for vehicle queue length measurement and peak-hour/ off-peak traffic. In order to simulate measurement errors from sensors in traffic engineering practice, we introduced gaussian noise for queue length measurements so that the models were asked to make decisions based on deviated inputs. The peak-hour traffic environment was also introduced to test the models' robustness and stability when handling with different traffic densities. By changing the vehicle spawn rate, the models experience "peak-hour" traffic with doubled incoming traffic density for half of the period and "off-peak" traffic for the other half 5,6 .

5.2 Performance by introduce delay of the signal control - PPO & round-robin baseline

As observed in our round-robin baseline under a fixed environment, both the average waiting time and throughput exhibit a clear optimum around a 30-step phase duration: shorter phases reduce throughput, while longer phases increase waiting time. This suggests that introducing a controlled delay in signal switching might also benefit the PPO agent.

To test this hypothesis, we conduct an ablation study where we explicitly vary the minimum delay between signal switches. We sample several delay values (e.g., 0, 10, 30, 45 and 60 steps) and train evaluate the PPO agent under each configuration. 0-step delay corresponds to the vanilla PPO setup without any enforced extra delay on signal switching.

The overall performance trend of PPO closely follows the pattern observed in the round-robin baseline, which matches our expectation: moderate delays improve throughput up to a point, after

which excessive delay begins to hurt waiting time. As illustrated in Figure 8, a delay of 30 steps provides the best trade-off: The PPO agent's throughput at 30-step delay is higher than the round-robin baseline (approximately 0.20 vs. 0.18). The average waiting time at 30-step delay remains low (around 50 steps), comparable to the baseline's minimum.

These results indicate that enforcing a moderate signal-switch delay helps PPO exploit phase stability for higher throughput, while still maintaining low waiting times.

5.3 Performance on Different Reward Function Composition & Configuration

In the RL models, the composition and configuration of reward functions are directly associated with the outcoming model performance. In this case, we designed a number of reward functions with differences in key components composition and weight to compare their influence on performance. Detailed reward function compositions and results are shown in Appendix 12.

6 Discussion

In general, compared with the baseline fixed-timing model, DQN outperforms significantly with higher average throughput and lower overall waiting. It seems unintuitive that the models are also performing better in a realistic environment than the ideal one, and the reason is that when the traffic density is high, the traffic throughput potential is fully exploited and the importance of having a good policy is emphasized. In an ideal environment, with low traffic and short queues, even a poor strategy will not perform badly. However, in a realistic environment with heavy traffic and congestion, a good strategy can directly reduce waiting time considerably. In round-robin experiments, we found that phase duration has a strong impact on performance. This motivated our PPO ablation with different minimum delays. PPO shows a similar trend: a moderate delay gives the best trade-off between throughput and waiting time, while very small or very large delays hurt performance.

Comparing the model performances trained with different reward functions, the stable-incremental version outperforms other models. It reduces the reliance on instant throughput improvement, which avoids oscillations in reward computation and encourages the model to learn long-term patterns. As a result, it provides clearer optimization goals and easier convergence for the model. On the contrary, some criteria that other reward functions implemented are not suitable for this experiment. The average wait, for example, is a highly lagging indicators, which makes the model hard to associate action and the instant reward. Similar phenomena were observed in both DQN and PPO reward shaping experiments.

Hence, the balance between throughput, delay, and fairness is not automatically discovered. It is largely determined by what we choose to measure and penalize in the reward function. Reward components such as balanced queue penalties or pure delay terms can be useful, but if they dominate the signal, they may fail to train a policy that both maximizes throughput and minimizes average waiting time, as illustrated by balance queue strategy. The Stable-incremental reward provides a good balance by combining queue length, accumulated delay, and steady flow incentives.

7 Conclusions

This project studied the application of RL for adaptive traffic control at urban intersections. By comparing DQN and PPO with traditional baselines, we demonstrated that learning-based controllers can significantly reduce waiting time and increase throughput in dynamic conditions. Through ablation studies, we showed that signal-switch delay, reward function design, and environment realism deeply influence training and performance. Overall, these results indicate the potential of deep reinforcement learning as an effective approach for traffic control, while highlighting the importance of environment modelling and reward design.

References

[1] Y. Dong, T. Datema, V. Wassenaar, C. T. Kopar, and H. Suleman, "Comprehensive training and evaluation on deep reinforcement learning for automated driving in various simulated driving maneuvers," 2023, accessed: 2025-11-04. [Online]. Available: https://arxiv.org/abs/2306.11466

[2] Yinyoupoet, "A detailed explanation of deep q-networks (dqn) in deep reinforcement learning," https://yinyoupoet.github.io/2020/02/18/%E6%B7%B1%E5%BA%A6%E5%BC%BA%E5%8C%96%E5%AD%A6%E4%B9%A0%E4%B9%8B%E6%B7%B1%E5%BA%A6Q%E7%BD%91%E7%BB%9CDQN%E8%AF%A6%E8%A7%A3/, 2020, accessed: 2025-11-06.

[3] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, and N. de Freitas, "Dueling network architectures for deep reinforcement learning," *arXiv preprint arXiv:1511.06581*, 2015.

Appendix

1-Environment Illustration

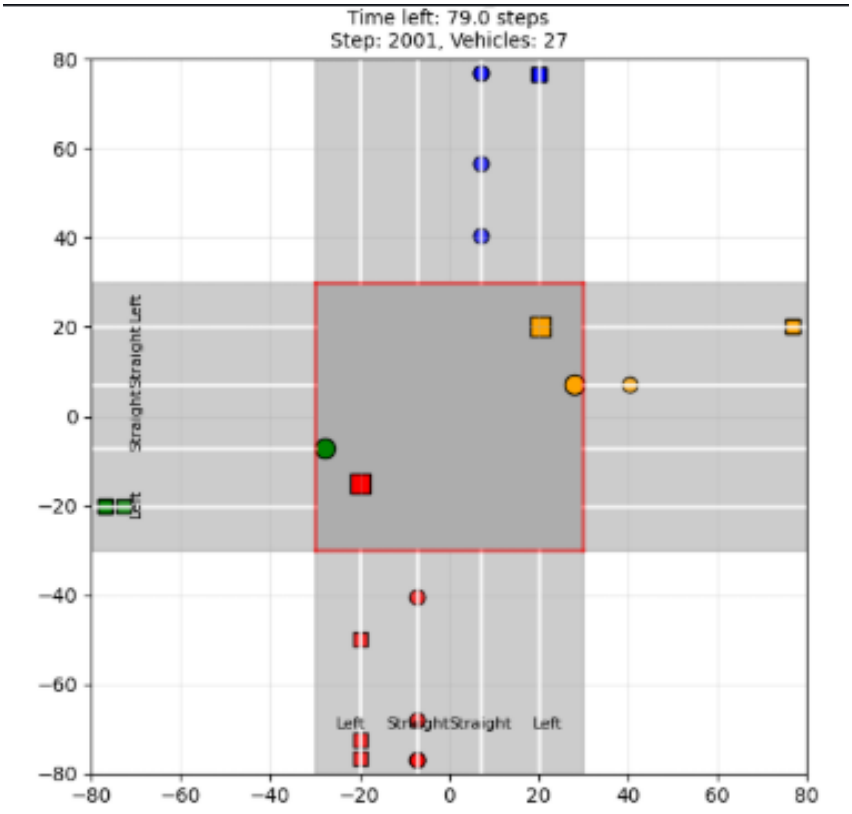

Figure 1: Traffic Environment

2-Baseline Example Illustrations

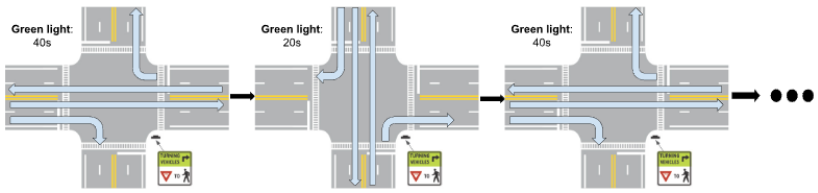

Figure 2: Example: a Fixed-Time cycle contains 60s. 40s for East-West. 20s for South-North

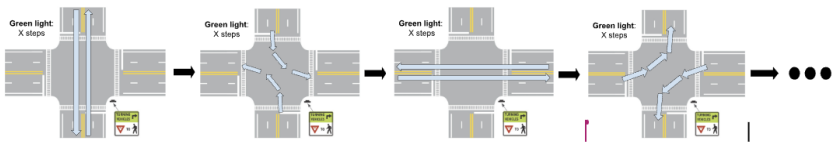

Figure 3: Example: a Round-Robin cycle switches phase after X steps

3-Network Illustration

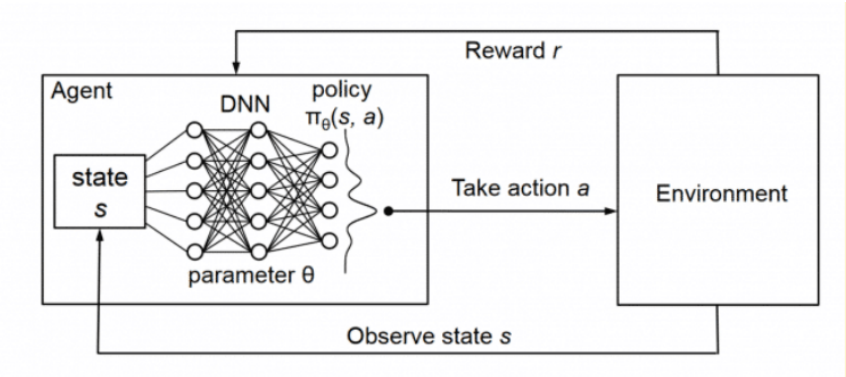

Figure 4: Deep-Q Network (2)

4-Results comparison

Table 1: Performance on Different Reward Function Composition & Configuration

Reward Function	Composition	Explanation
Default	- Penalizes large queues, waiting time rate; - Rewards the improvement relative to the last step, absolute throughput	- Reactive - Good for exploring fast, high-moving pattern - Noisy
Stable-Incremental	- Penalizes strongly for queue length, accumulating delay - Rewards steady traffic flow	- More stable training & less prone to oscillations - Works better in "realistic" environments (peak/off-peak)
Delay-oriented	- Penalizes delay per vehicle strongly; - Rewards basic flow maintenance	- Prioritizes minimizing delay - Might slow down peak flow for fairness
Balanced-queue	- Penalizes when one direction has a very long queue compared to others	- Encourages balanced traffic flow - But delay distribution is much more even - Avoids starving minority lanes

Table 2: Comparison of DQN and PPO results under different reward functions

Reward	**DQN Result**		**PPO Result**	
Function	**Throughput**	**Waiting Time**	**Throughput**	**Waiting Time**
Default	0.1079	149.87	0.189	104.039
Stable	0.1472	92.08	0.18	81.881
Delay-oriented	0.0955	115.19	0.173	86.592
Balanced-queue	0.1185	123.23	0.077	236.376

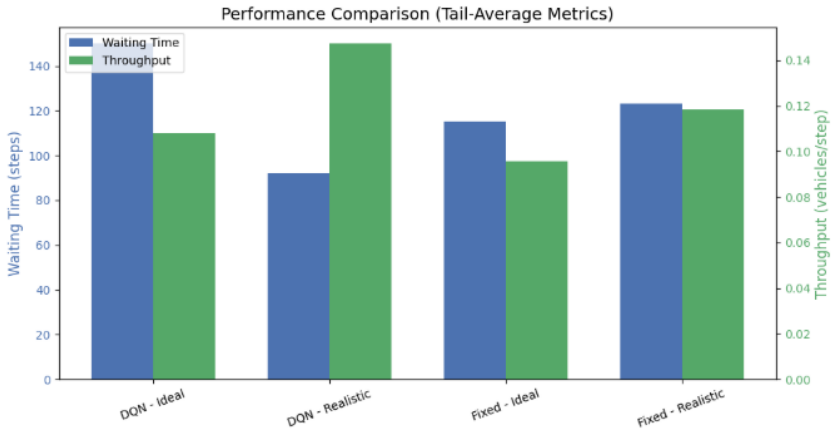

Figure 5: Performance Comparison (DQN & FT)

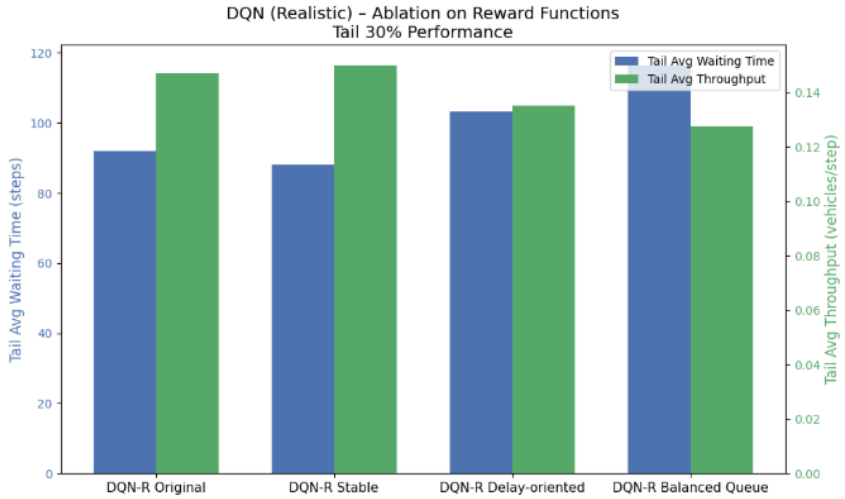

Figure 6: DQN (Realistic) - Ablation on Reward Functions

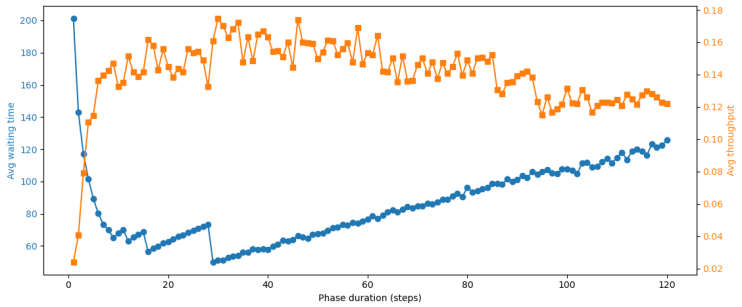

Figure 7: Round-Robin baseline performance switches per fixed number of x-steps

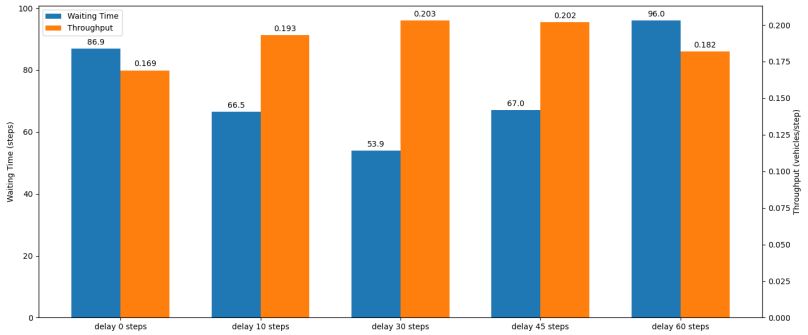

Figure 8: PPO model performance after introducing signal delay

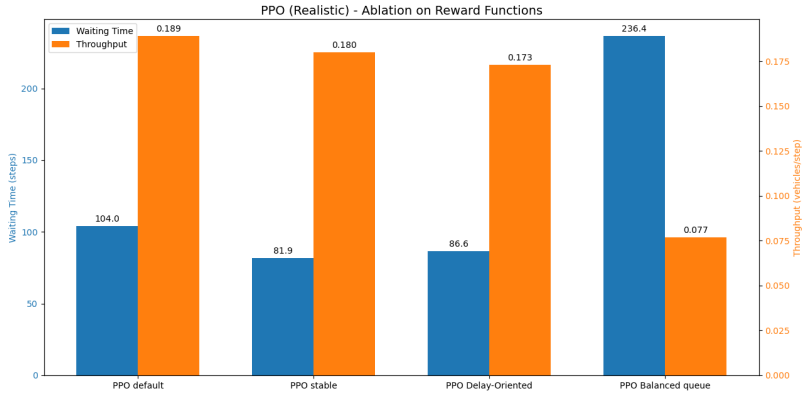

Figure 9: PPO (Realistic) - Ablation on Reward Functions

5-Pseudo code

Initialize actor parameters θ and critic parameters ϕ

Collect trajectories by running π_θ in the environment:

for episode $= 0, 1, \ldots, N$ do

  for $t = 0, 1, \ldots, T-1$ do

    observe state s_t

    sample action $a_t \sim \pi_\theta(\cdot \mid s_t)$

    apply action, receive reward r_t and next state s_{t+1}

  end for

  compute returns R_t and advantages $\hat{A}_t$ using GAE:

  for $t = 0$ up to $T-1$ do

    $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$

    $\hat{A}_t = \delta_t + \gamma\lambda\hat{A}_{t+1}$

    $R_t = r_t + \gamma R_{t+1}$

  end for

  update actor and critic using K epochs of minibatch SGD:

for each minibatch B do

compute $L_{CLIP}(\theta)$ on B

compute $L_{value}(\phi)$ on B

compute $L_{entropy}(\theta)$ on B

maximize combined loss:

$L(\theta, \phi) = L_{CLIP}(\theta) - c_v L_{value}(\phi) + c_{entropy} L_{entropy}(\theta)$

end for

end for

Contest for Information Sharing

As part of this course, we may share selected project materials (e.g., reports, presentation slides, and presentation recordings) on the course webpage as learning resources for future students. Additionally, we may use anonymized project information for internal statistical analysis of course outcomes. Please indicate your preferences below. *Your choices will not affect your grade in any way.*

Consent for Sharing Project Materials

Please keep the item you wish and remove the other one.

- All group members consent to allow the project materials (report, slides, and presentation recording) to be shared on the course page for future students.

Consent for Use of Project Information in Statistical Analysis

Please keep the item you wish and remove the other one.

- All group members consent to allow anonymized information about the project (e.g., topic, methods, outcomes, grades) to be used by the instructor for statistical analysis and course improvement.

Optional Comments

Please list any specific conditions or comments here.

Group Identification

- Group number: G7
- Names of group members: Mingjie Gao, Xiao Sun, Tianrui Luo, Zhiqiang Yin
- Signature of of group members: Mingjie Gao, Xiao Sun, Tianrui Luo, Zhiqiang Yin
- Date: 2025/12/07